Архитектура ЭВМ

Курс «Информатика», лекция 3

Развитие ЭВМ

Годы	Технология	Авторы	Технологичская база
17-й век	Арифмометр	Вильгельм Шиккард, Блез Паскаль, Лейбниц	Механические операции.
18-й век	Машина Беббиджа	Чарльз Беббидж	Механические операции
1890-й год	Табулятор (электро-механическая счётная машина).	Герман Холлерит, основатель CRT (в последствии IBM)	Реле
1950-е годы	Первое поколение ЭВМ	Фон Нейман.	Лампы
1960-е годы	Второе поколение ЭВМ		Транзисторы
1970-е годы	Третье поколение ЭВМ		Интегральные схемы
1980-е годы	Четвёртое поколение ЭВМ		Сверхбольшие интегральные схемы
1990-е годы до наших дней	Пятое поколение ЭВМ		Новые физические эффекты, нейронные сети…

Принципы фон Неймана

Процессор – устройство предназначенное для выполнения машинных команд.

ПРОГРАММА УПРАВЛЯЕТ РАБОТОЙ ПРОЦЕССОРА!

Термины, обозначающие процессор:
- **ЦПУ** (центральное процессорное устройство).
- **ЦП** (центральный процессор.
- **CPU** (central processing unit).

Архитектура процессора – схема соединения составляющих его блоков и принципы их взаимодействия.
Архитектура современных процессоров основана на принципах, сформулированных фон Нейманом в 1946 году.

1) **Принцип двоичного кодирования.**
2) **Принцип последовательного программного управления.**
3) **Принцип однородности памяти.**
4) **Принцип адресуемости памяти.**
5) **Принцип условного перехода.**

Блоки процессора

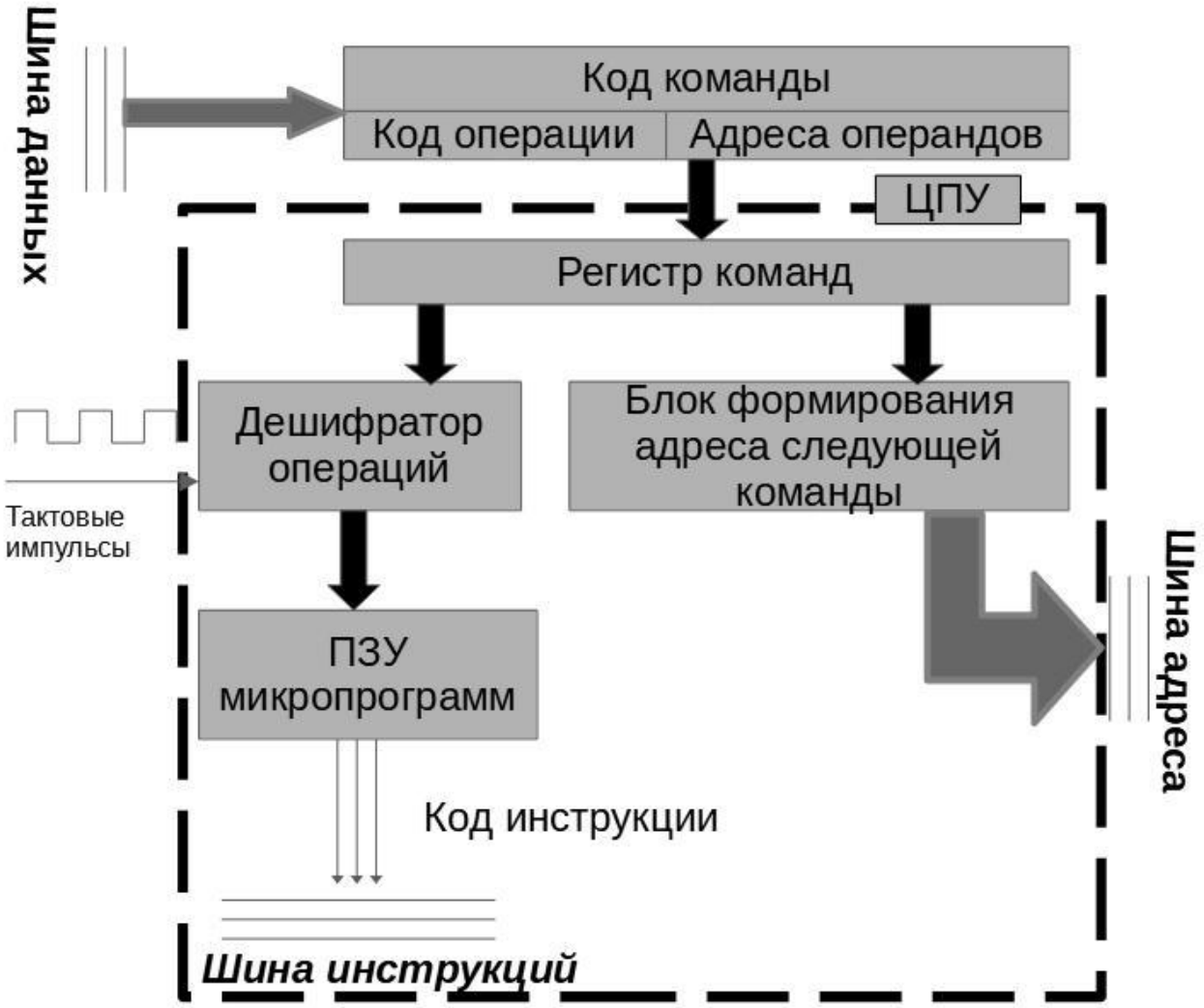
В состав процессора, соответствующего принципам фон Неймана входят 4 блока:

- **арифметико-логическое устройство;**
- **устройство управления;**
- **микропроцессорная память;**
- **порты ввода/вывода.**

Шины процессора:

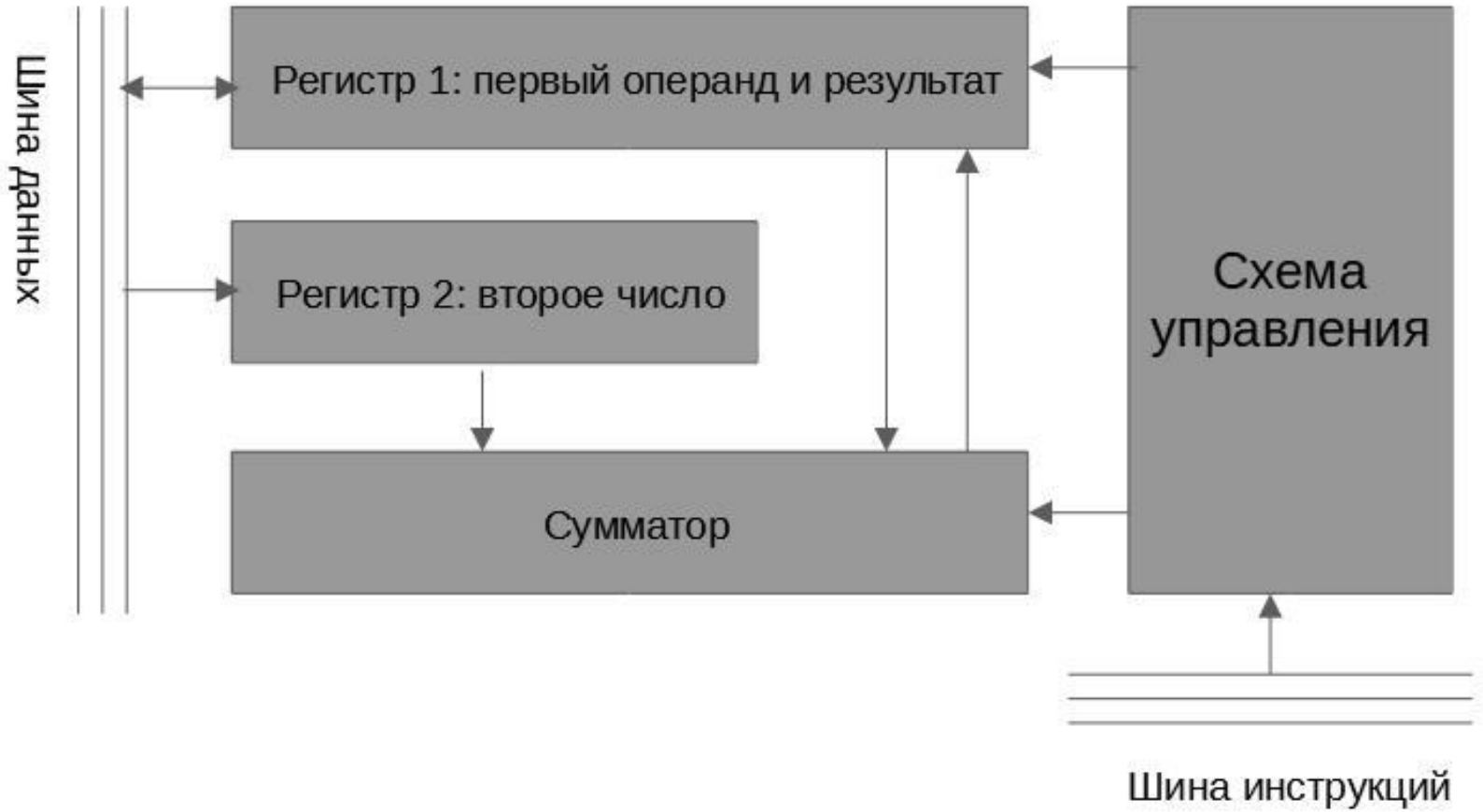
- **шина адреса;**
- **шина данных;**
- **шина управления;**
- **шина прерываний.**

Устройство управления

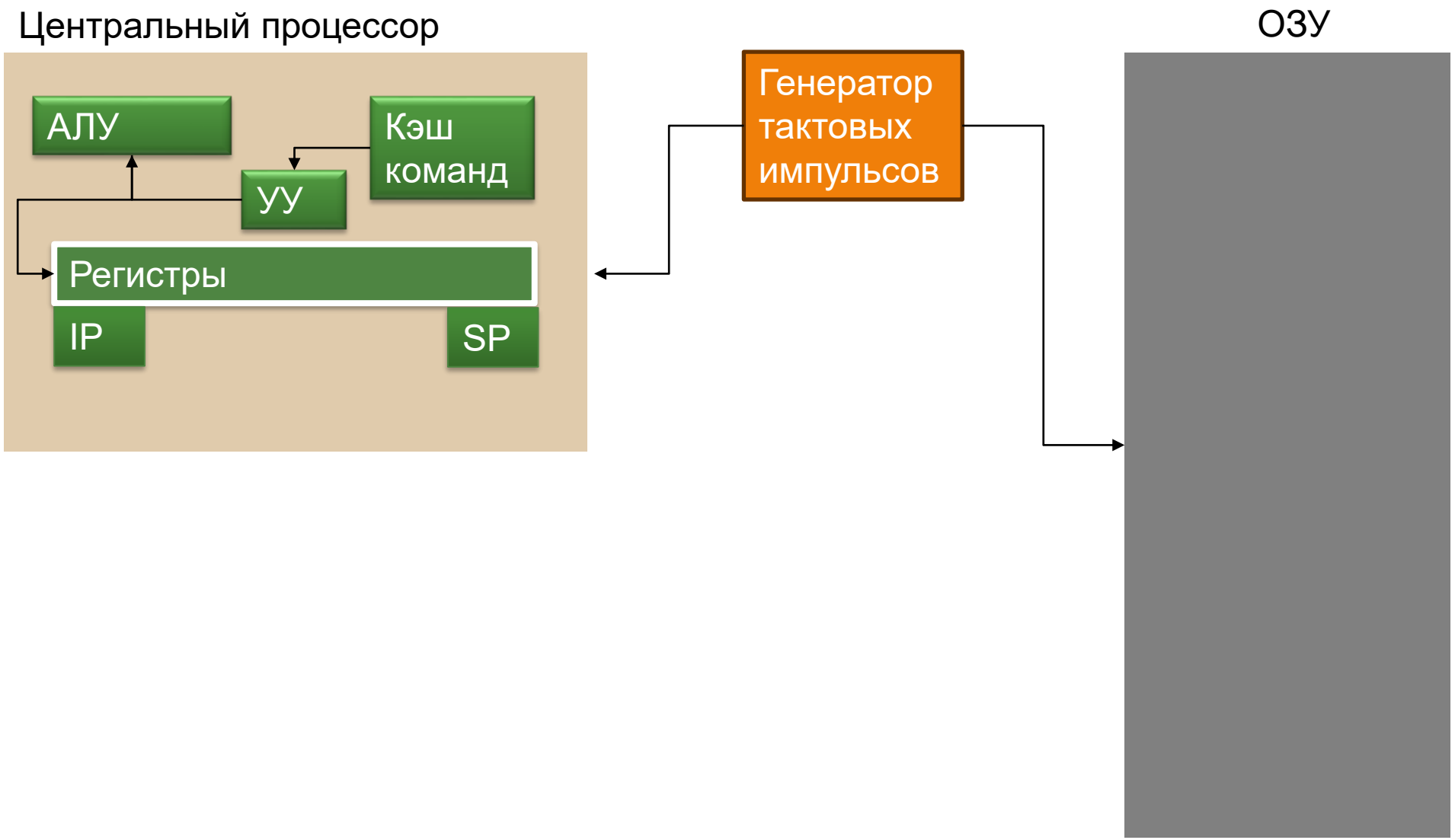


Арифметико-логическое устройство

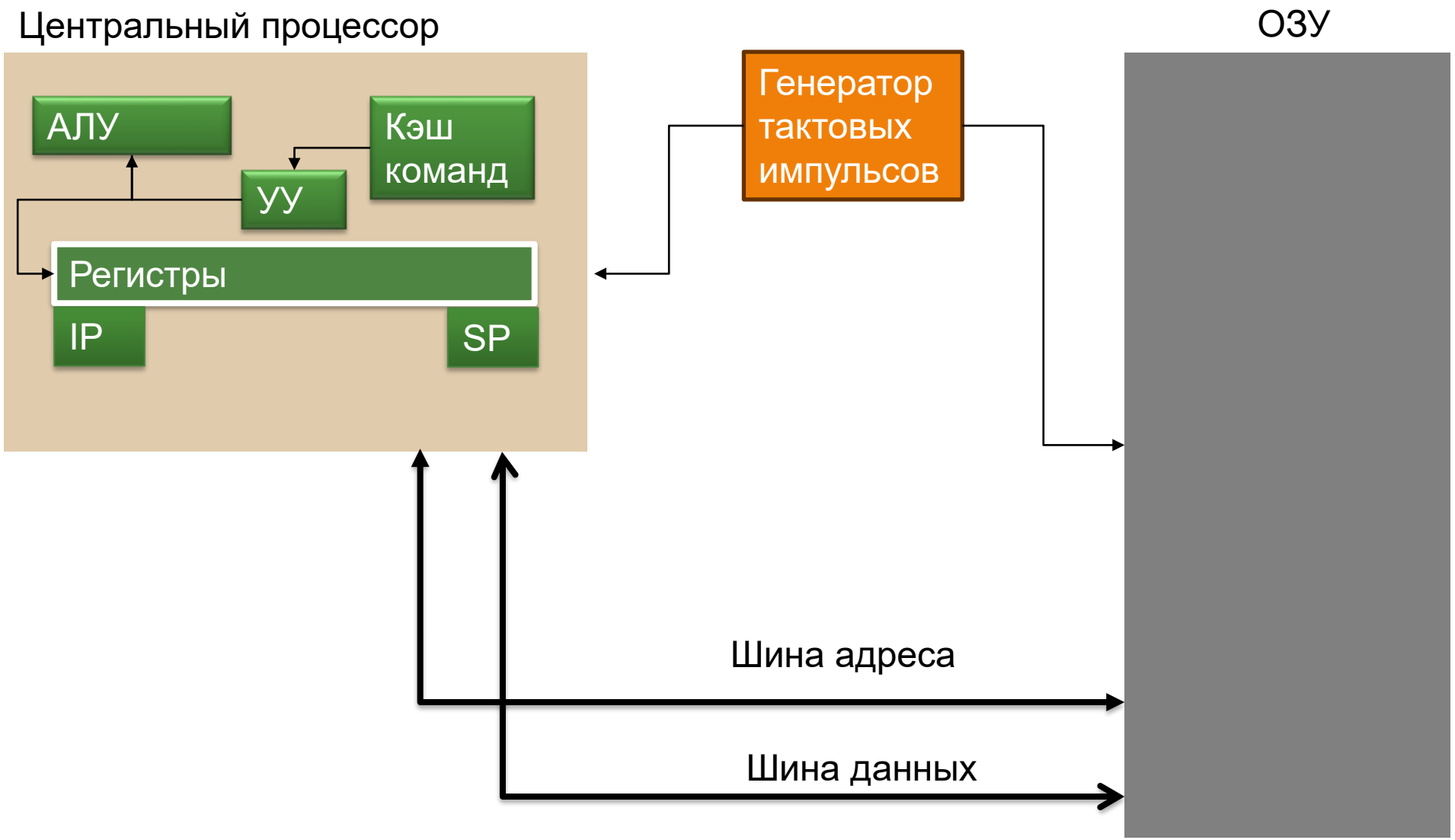
Длина регистра 1 в два раза больше длины регистра 2



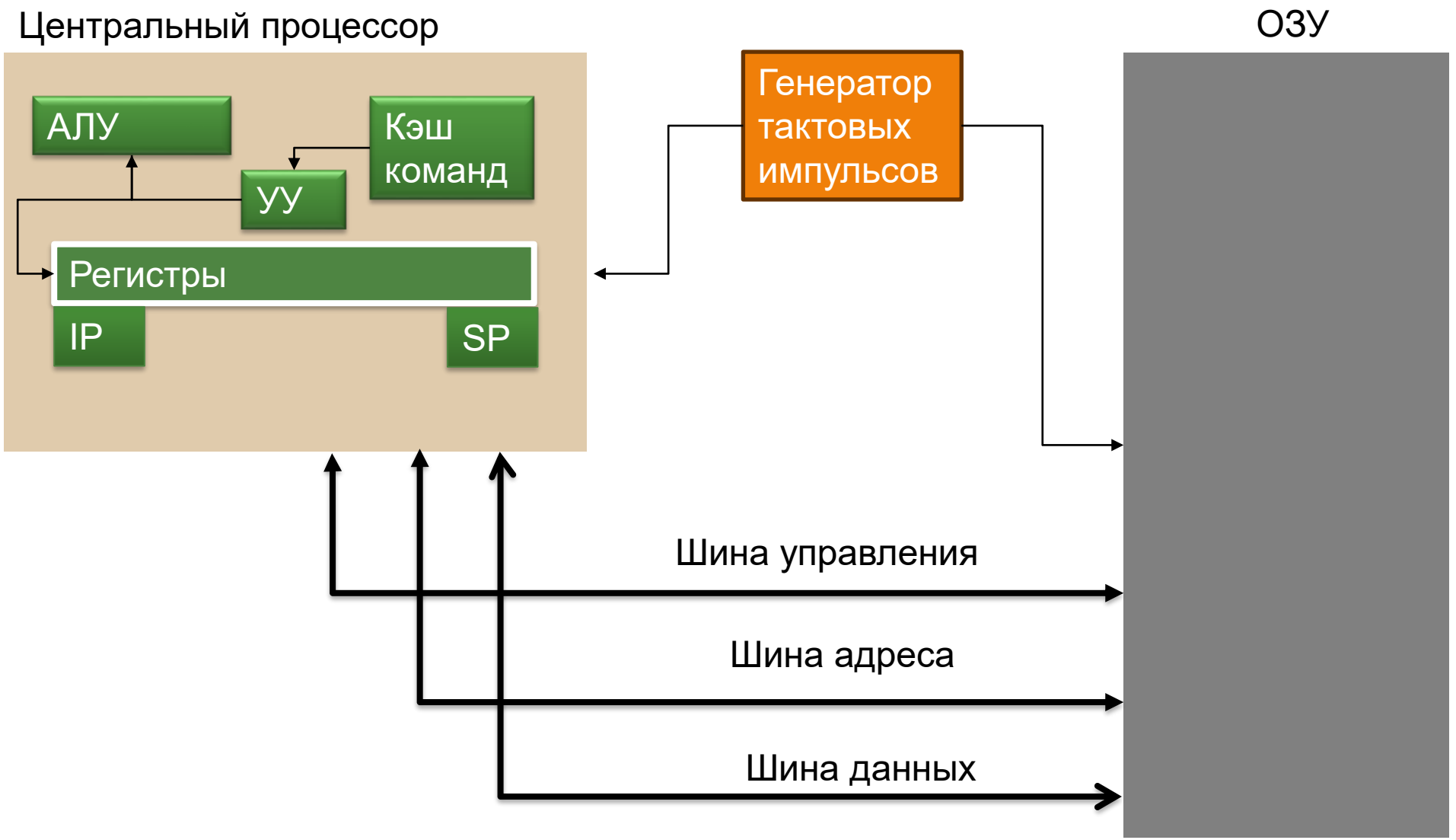
Архитектура ЭВМ



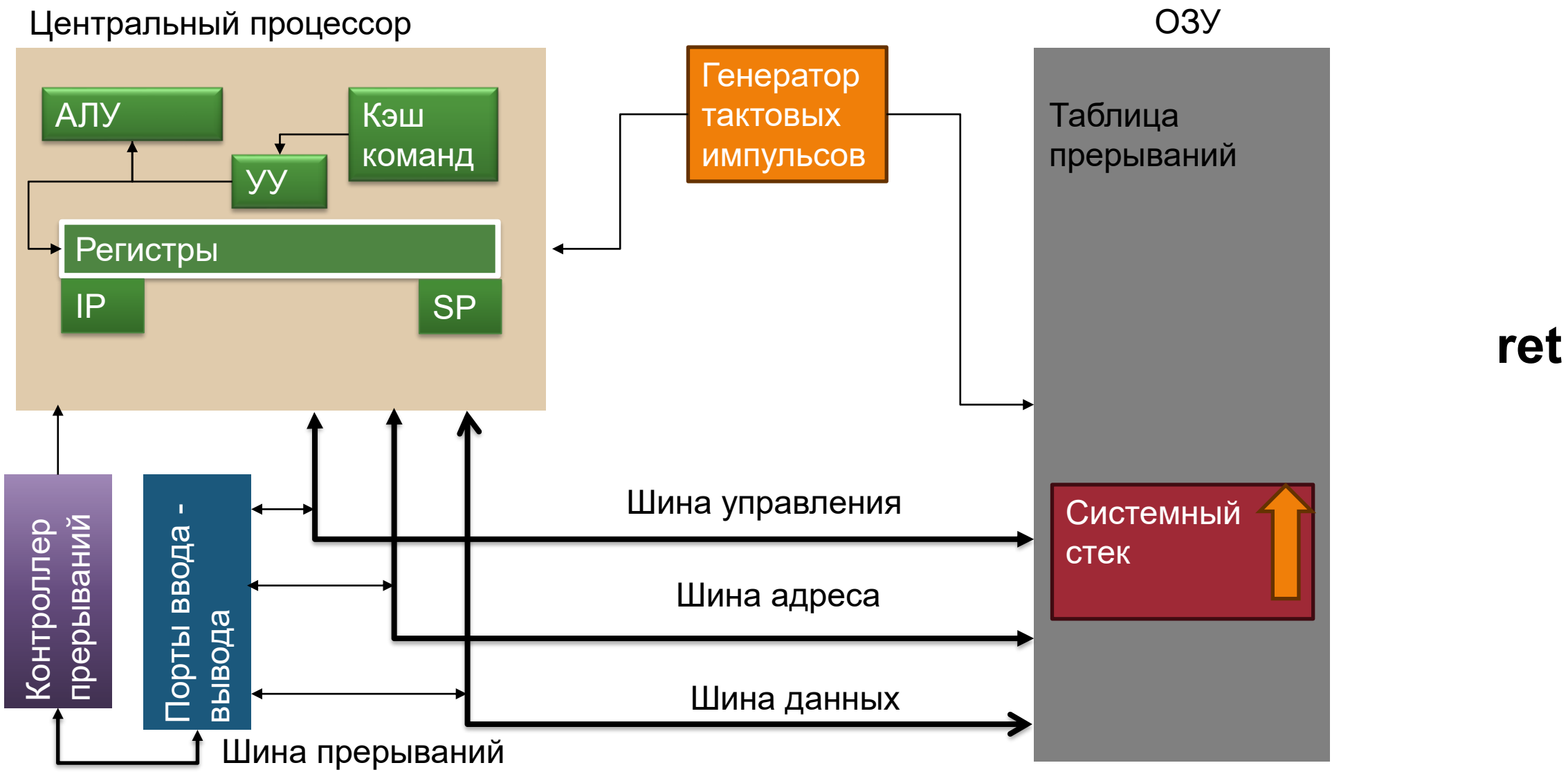
Архитектура ЭВМ



Архитектура ЭВМ

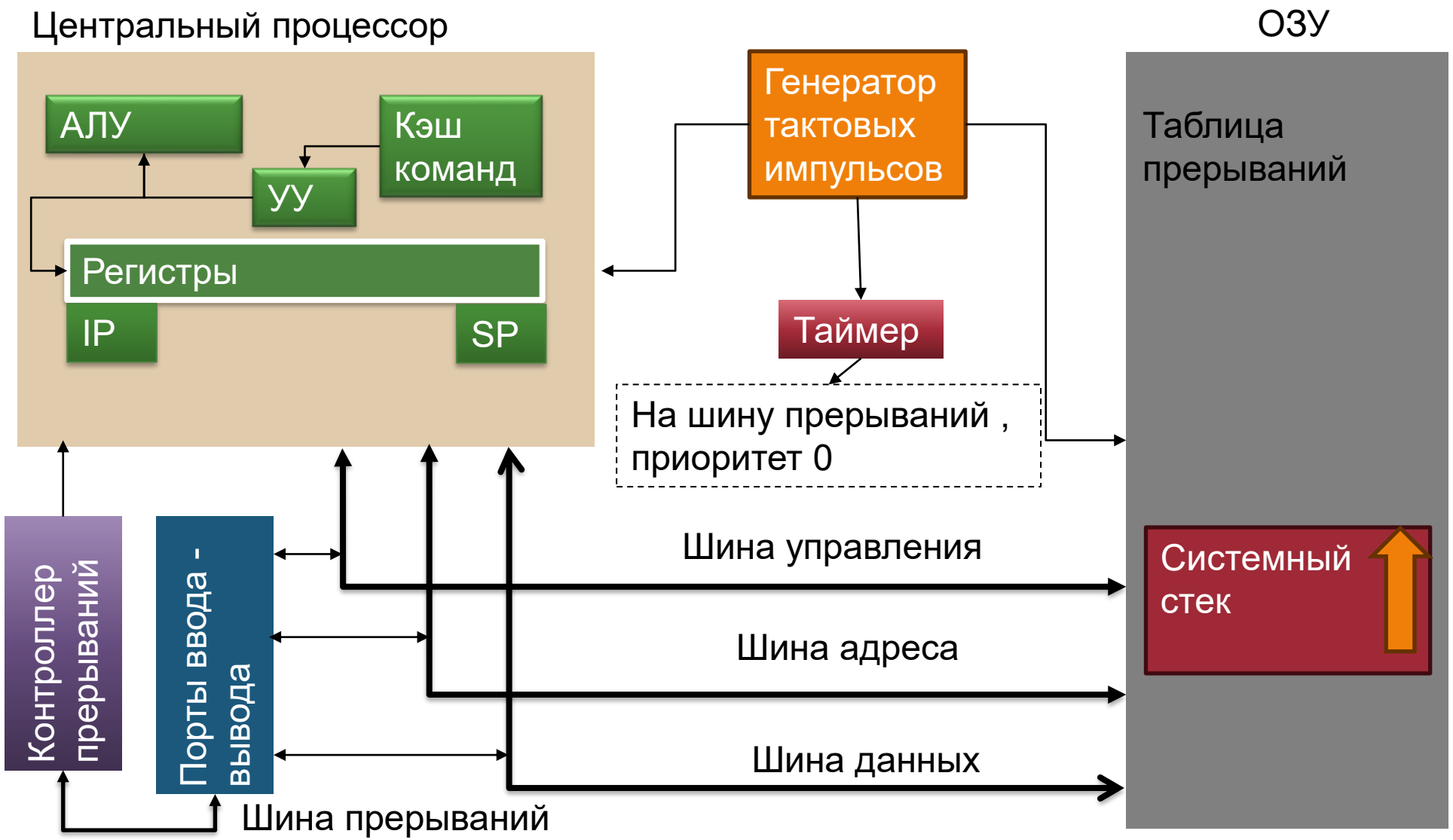


Архитектура ЭВМ



ret

Архитектура ЭВМ



Микропроцессорная память и порты ввода/вывода

Микропроцессорная память – сверхбыстродействующая память, состоящая из регистров процессора.

Регистры делятся на:
- регистры общего назначения;
- специальные регистры.

Порты ввода/вывода состоят их двух регистров:
- регистр данных;
- регистр управления.

Порт ввода/вывода может находится в одном из трёх состояний:
- приём данных;
- передача данных;
- изоляция шины (состояние с высоким импедансом).

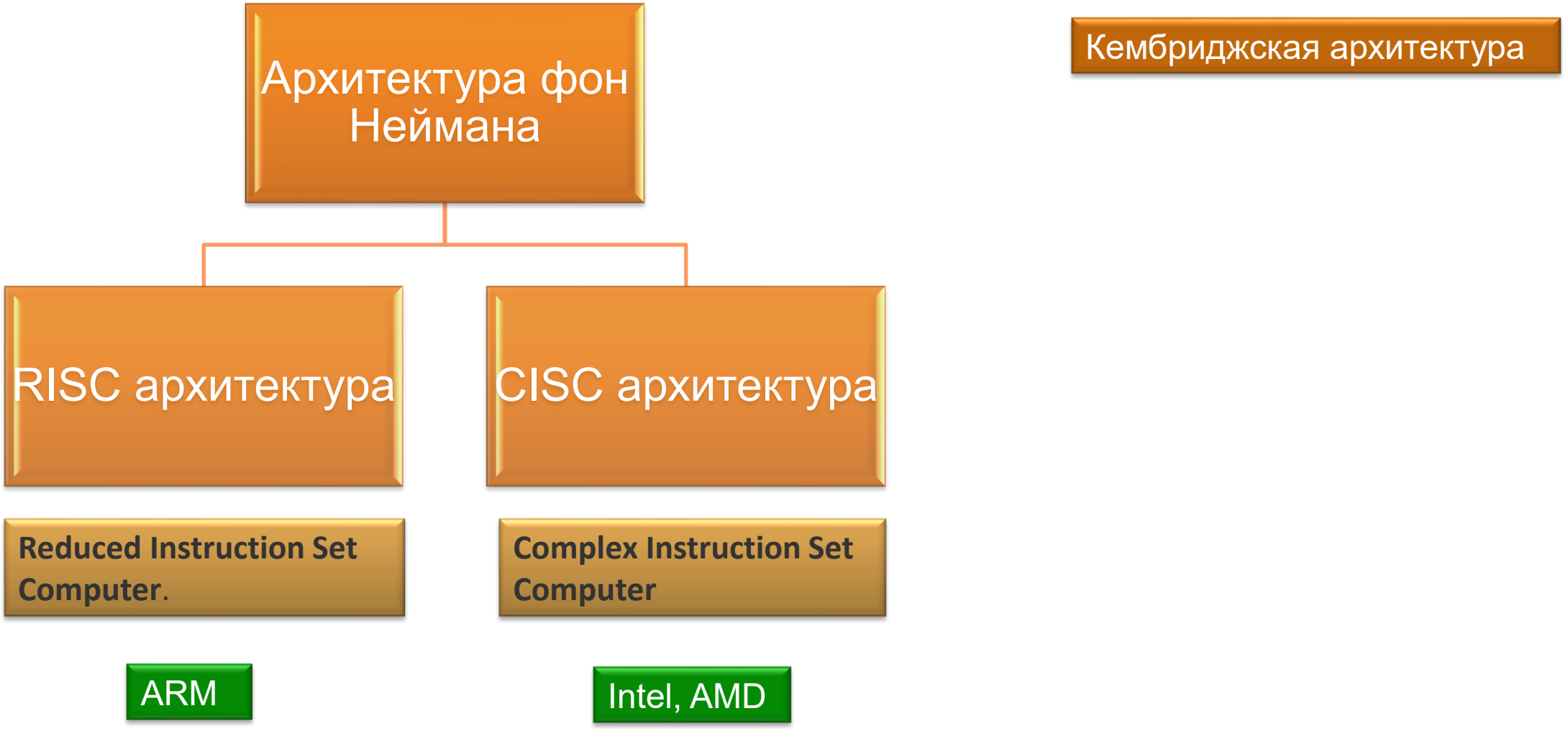
Порты ввода/вывода подключены к шинам адреса, данных и управления.

Пример выполнения команды процессора

Абстрактная команда СУМ 11111 22222 суммирует число, находящееся в ячейке оперативной памяти 11111 с числом, находящимся в ячейке оперативной памяти 22222.

- **Первый такт.** Пересылка команды из оперативной памяти в регистр команд.
- **Второй такт.** Считывание данных из ячейки 11111 оперативной памяти и запись их в регистр 1 АЛУ.
- **Третий такт.** Считывание данных из ячейки 22222 оперативной памяти и запись их регистр 2 АЛУ.
- **Четвёртый такт.** Сложение чисел в АЛУ и запись результата в регистр 1.
- **Пятый такт.** Запись данных из регистра 1 в ячейку 11111 оперативной памяти.

Современные архитектуры процессоров



Операции и операторы

Операция - процесс, который длится конечное время и приводит к желаемому, заранее определённому результату. Описывает **КАК** надо сделать.
Операция совершается с одним или несколькими объектами, состояние которых изменяется в ходе её выполнения.

Оператор – символьное обозначение операции. Описывает **ЧТО** надо сделать.

Операнд – обозначение объекта, участвующего в операции.

Операторы делятся на:
- простые;
- составные.

Арифметическое выражение – составной оператор.
Старшинство арифметических операторов:
1) Умножение, деление;
2) Сложение, вычитание.

Пример: 2 + 3 * 7 - 9

Операции и операторы

Логическое выражение – составной оператор.

Старшинство логических операций:

1) Отрицание;
2) Логическое И
3) Логическое ИЛИ

Пример: not V and L or Z and Y

1) not V (операнд 1)
2) ***операнд 1*** and L (операнд 2)
3) Z and Y (операнд 3)
4) ***операнд 2*** or ***операнд 3***

Операции и операторы

Разложение оператора на операции

Пример 1, оператор сложения
вычислить 5 + 7
Команда процессора 1: R1 := 5
Команда процессора 2: R2 := 7
Команда процессора 3: R1 := R1 + R2

Пример 2, оператор умножения без сопроцессора
вычислить 7 * 5
Команда процессора 1: R1 := 0
Команда процессора 2: R2 := 7
Команда процессора 3: R3 := 5
Команда процессора 4: Пока R3 больше 0 выполнять команды 5 и 6:
Команда процессора 5: R1 := R1 плюс R2
Команда процессора 6: R3 := R3 минус 1

Операции и операторы

Пример 3, оператор умножения без сопроцессора

вычислить 5 * 7

Команда процессора 1: R1 := 0

Команда процессора 2: R2 := 5

Команда процессора 3: R3 := 7

Команда процессора 4: Пока R3 больше 0 выполнять команды 5 и 6:

Команда процессора 5: R1 := R1 плюс R2

Команда процессора 6: R3 := R3 минус 1

Пример 3, оператор умножения с сопроцессором

вычислить 5 * 7

Команда процессора 1: R1 := 5

Команда процессора 2: R2 := 7

Команда процессора 3: R1 := R1 умножить на R2

Оператор присваивания

Переменная - символьное обозначение ячейки памяти.
Значение переменной – содержимое ячейки памяти.

Значение переменной нельзя «стереть», его можно только изменить.

Оператор присваивания обозначается **=** или **:=**

Оператор присваивания выполняется за один такт процессора.

Оператор присваивания имеет два операнда: **левая часть** и **правая часть**.

Особый случай оператора присваивания: в выражении, стоящем в правой части, используется переменная, стоящая в левой части.

Условные операторы

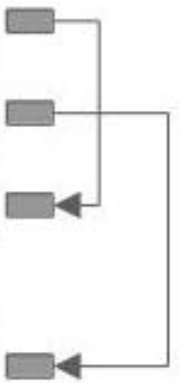
Задача: вычислить значение выражения **C := A / B**

Что произойдёт, если B = 0?

Пятый принцип фон Неймана:

принцип последовательного программного управления, может нарушаться для перехода к произвольной команде в последовательности команд.

Номер ячейки ОЗУ	Содержание команды
1000	Проверка на равенство B нулю.
1001	Если B равно нулю, то перейти к ячейке 1003
1002	Если B не равно нулю, то перейти к ячейке 1005
1003	Вывести на экран сообщение об ошибке.
1004	Остановить выполнение программы.
1005	Продолжить вычисления.



Условные операторы

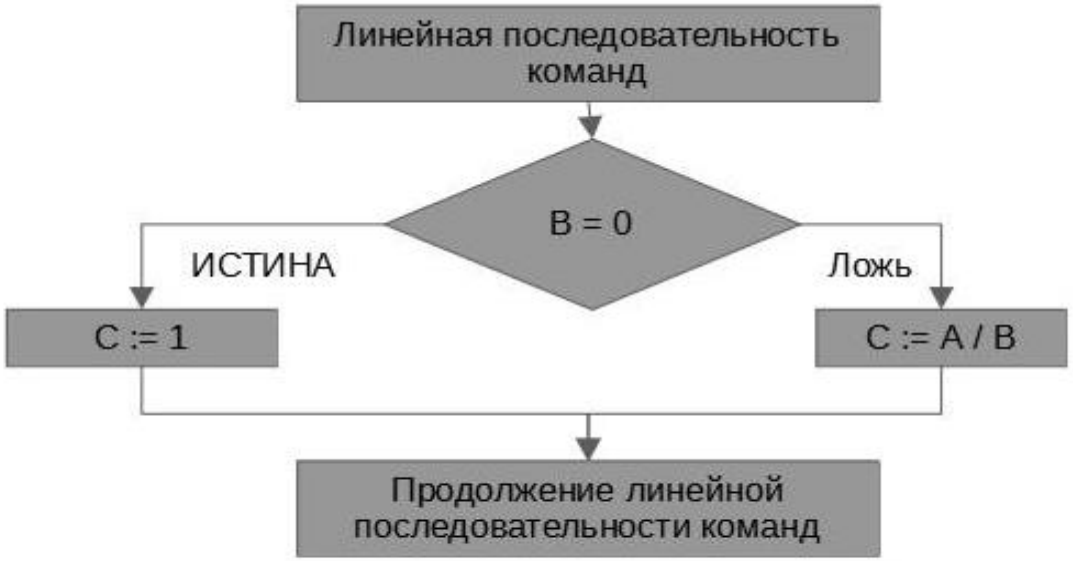
Безусловный переход осуществляется путём записи в счётчик команд адреса произвольной ячейки памяти.
Оператор, реализующий команду безусловного перехода обозначается **GoTo**.

Оператор условного перехода обозначается **IF.**



Условные операторы

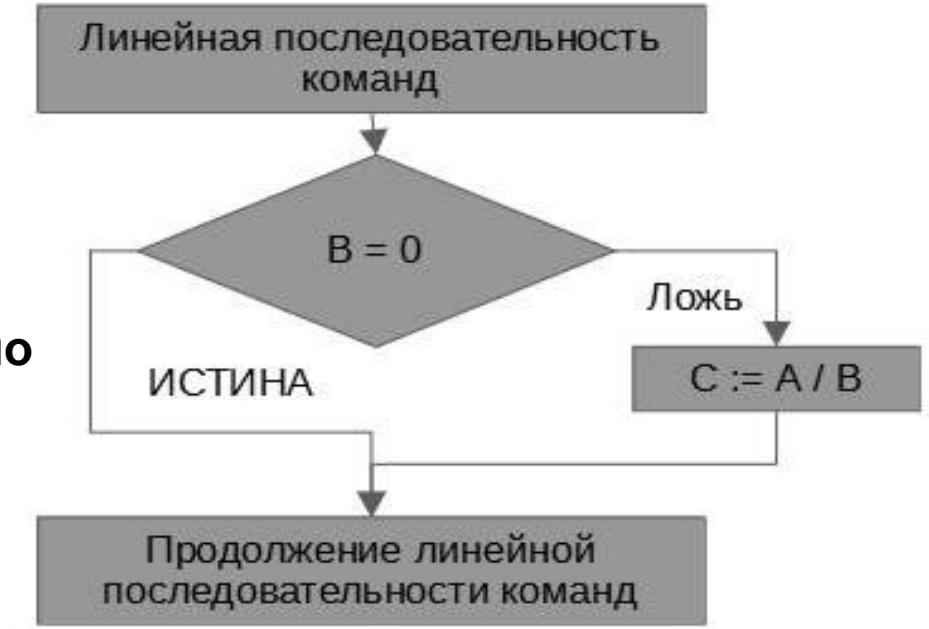
Если B = 0, то C := 1



Номер ячейки ОЗУ	Содержание команды
1000	Проверка на равенство B нулю
1001	Если B равно нулю, то переход к ячейке 1003
1002	Если B не равно нулю, то переход к ячейке 1005
1003	C := 1
1004	Переход к ячейке 1006
1005	C := A / B
1006	Продолжить вычисления

Условные операторы

Если B = 0, то C вычислять не нужно

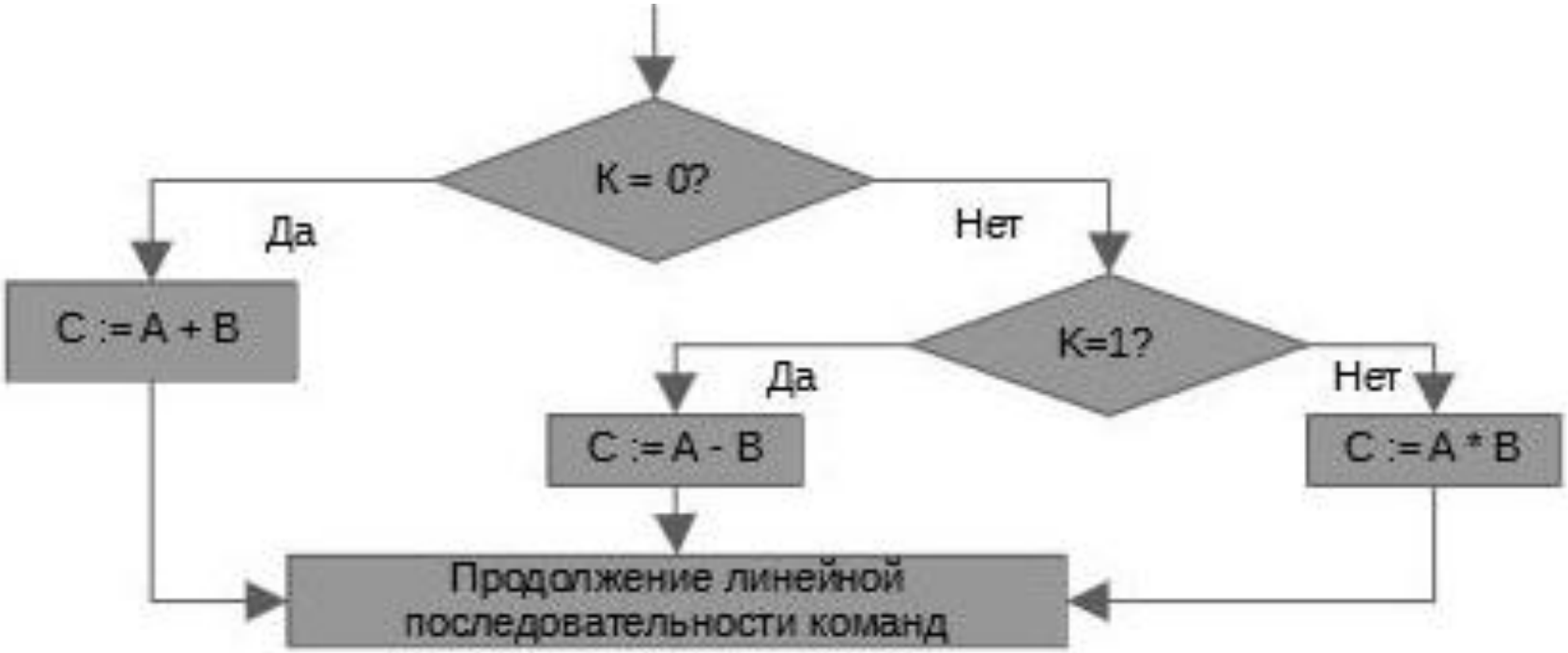


Номер ячейки ОЗУ	Содержание команды
1000	Проверка на равенство B нулю
1001	Если B равно нулю, то переход к ячейке 1004
1002	Если B не равно нулю, то переход к ячейке 1003
1003	C := A / B
1004	Продолжить вычисления

Условные операторы

Задача:
Заданы три переменные: К, С, А и В. Переменная К может принимать три значения: 0, 1, 2.
Если К = 0, то значение переменной С вычисляется по формуле С := А + В.
Если К = 1, то значение переменной С вычисляется по формуле: С := А – В.
Если К = 2, то значение переменной С вычисляется по формуле: С := А* В.

Оператор ветвления:
Switch или Case



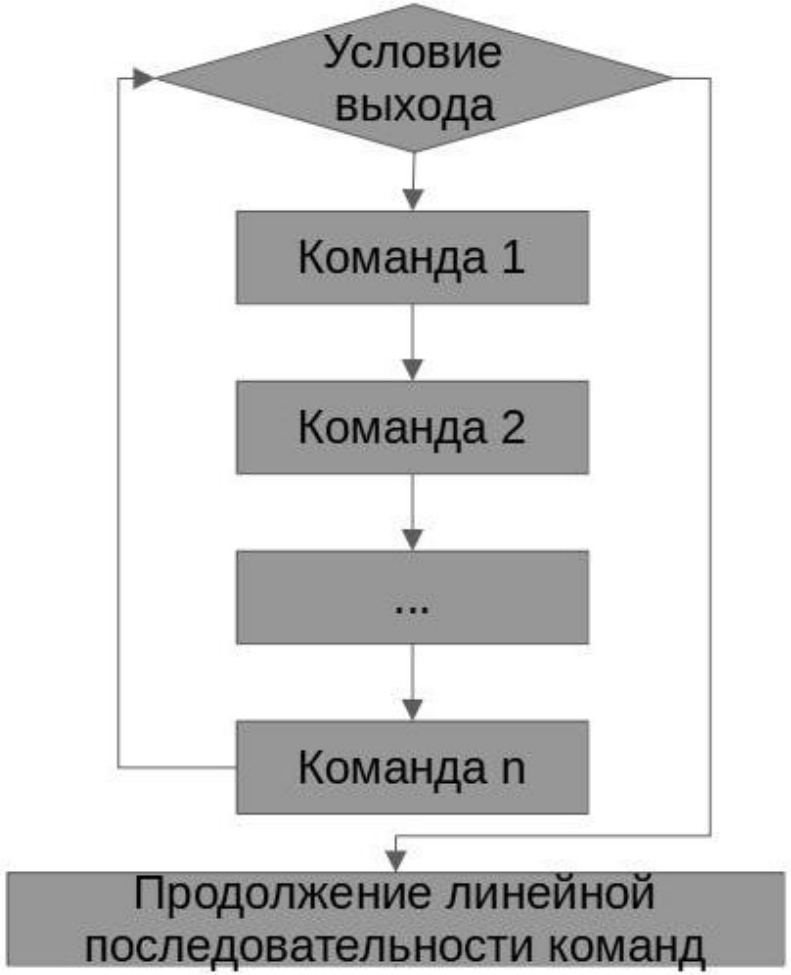
Оператор цикла

Передача управления вперёд: номер ячейки памяти, который заносится в регистр адреса процессора больше номера ячейки памяти с текущей командой.

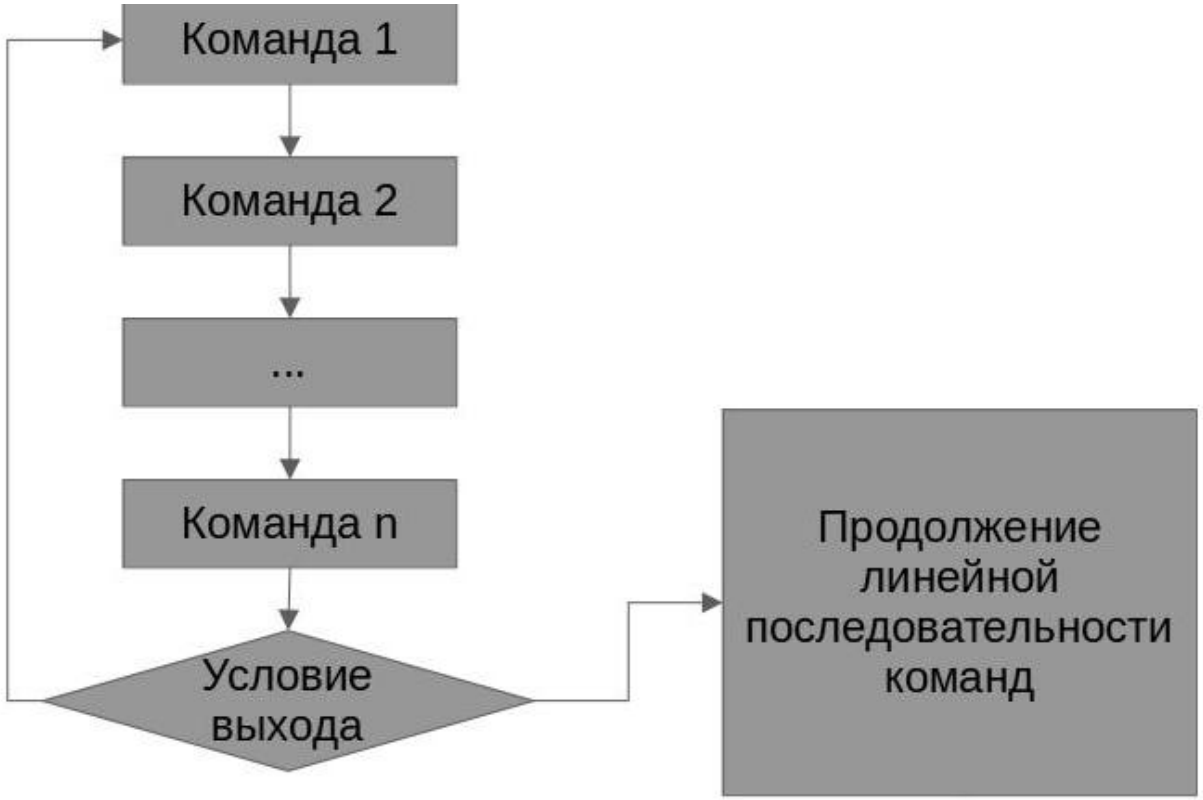
Передача управления назад: номер ячейки памяти, который заносится в регистр адреса процессора меньше номера ячейки памяти с текущей командой.



Оператор цикла



Цикл с предусловием



Цикл с постусловием

Оператор цикла

Структура цикла **WHILE**:
WHILE <условие продолжения цикла>
<Тело цикла>

Структура цикла UNTIL:
UNTIL
<Тело цикла>
<Условие продолжения цикла>

WHILE – универсальный оператор

Для того, чтобы циклы WHILE и UNTIL закончились необходимо, чтобы значение переменной, которая используется в условии продолжения цикла, изменялось в теле цикла.

```
C := 0
D := 7
L := 5
WHILE L > 0
        C := C + D
        L := L -1
```

Оператор цикла

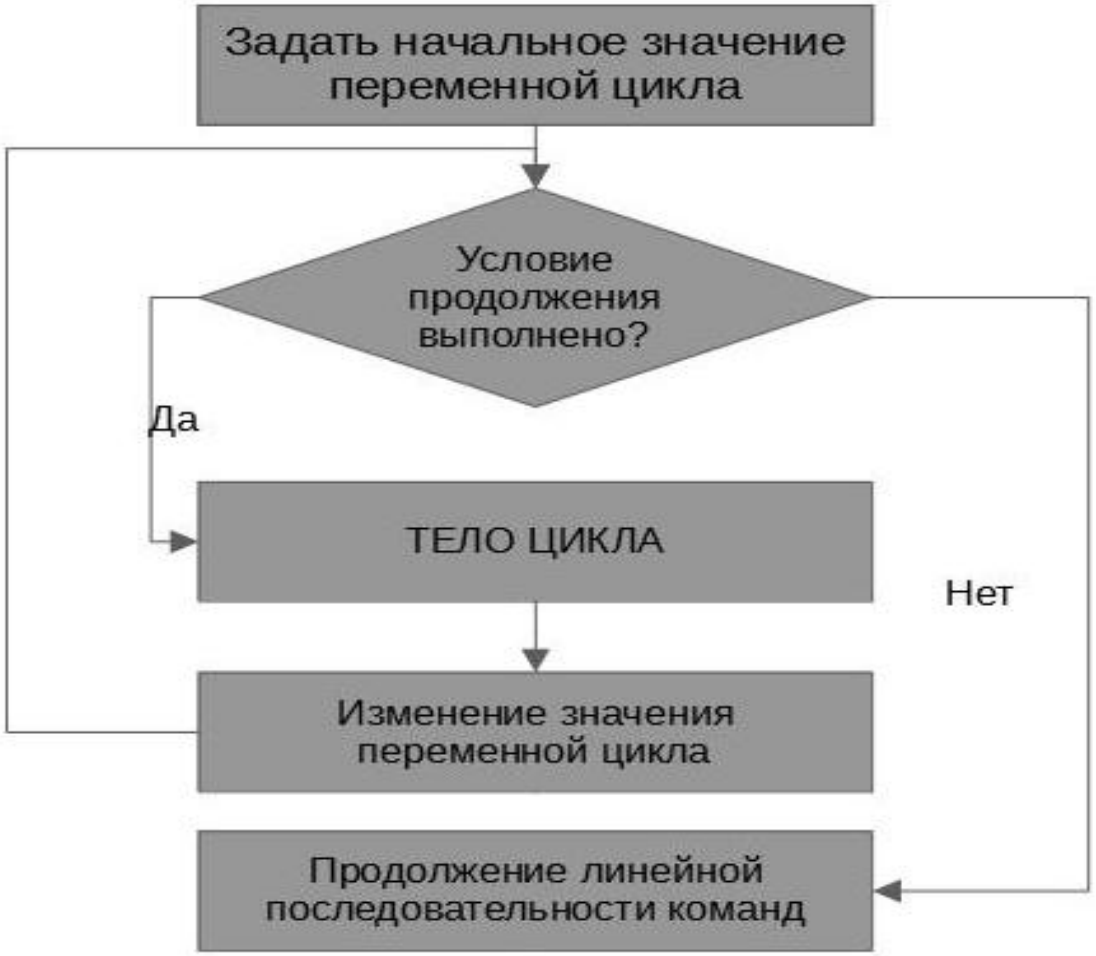
Цикл с параметром

Структура оператора цикла FOR:

ЗАГОЛОВОК ЦИКЛА

- переменной цикла присваивается начальное значение
- задаётся условие продолжения цикла, зависящее от переменной цикла
- задаётся выражение для вычисления нового значения переменной цикла

ТЕЛО ЦИКЛА



Операторы ввода и вывода

Ввод данных – это передача данных из внешнего устройства в оперативную память.
Вывод данных – это передача данных из оперативной памяти на внешнее устройство компьютера.

Поток ввода/вывода – часть операционной системы, помогающая процессору обмениваться данными с внешними устройствами.

Буфер обмена – блок оперативной памяти, используемый потоком ввода/вывода.

Потоки ввода/ вывода подразделяются на:

- **символьные**
- **бинарные**

Операторы ввода и вывода

Стандартные потоки ввода/вывода

- Стандартный поток ввода (обозначается cin, stdin) закреплён за клавиатурой компьютера.
- Стандартный поток вывода (обозначается cout, stdout) закреплён за экраном монитора.
- Стандартный поток ошибок (обозначается cerr, stderr) предназначен для вывода диагностических сообщений, закреплён за экраном монитора, но отделён от стандартного потока вывода.
- Стандартный поток печати (обозначение stdprn) закреплён за портом LPT1. Поскольку такой порт отсутствует в большинстве современных компьютеров, поток stdprn перенаправляют, например, на принтер, подключённый по шине USB.

Процессор не имеет команд для реализации обмена данными с внешними устройствами.

Кодировка символов

Кодировка ASCII (международный алфавит)

№ п/п	символ	двоичный код	№ п/п	символ	двоичный код	№ п/п	символ	двоичный код	№ п/п	символ	двоичный код
32	пробел	00100000	56	8	00111000	80	P	01010000	104	h	01101000
33	!	00100001	57	9	00111001	81	Q	01010001	105	i	01101001
34	"	00100010	58	:	00111010	82	R	01010010	106	j	01101010
35	#	00100011	59	;	00111011	83	S	01010011	107	k	01101011
36	$	00100100	60	<	00111100	84	T	01010100	108	l	01101100
37	%	00100101	61		00111101	85	U	01010101	109	m	01101101
38	&	00100110	62	>	00111110	86	V	01010110	110	n	01101110
39	'	00100111	63	?	00111111	87	W	01010111	111	o	01101111
40	(	00101000	64	@	01000000	88	X	01011000	112	p	01110000
41	)	00101001	65	A	01000001	89	Y	01011001	113	q	01110001
42	*	00101010	66	B	01000010	90	Z	01011010	114	r	01110010
43	+	00101011	67	C	01000011	91	[	01011011	115	s	01110011
44	,	00101100	68	D	01000100	92	\	01011100	116	t	01110100
45	-	00101101	69	E	01000101	93	]	01011101	117	u	01110101
46	.	00101110	70	F	01000110	94	^	01011110	118	v	01110110
47	/	00101111	71	G	01000111	95	_	01011111	119	w	01110111
48	0	00110000	72	H	01001000	96	'	01100000	120	x	01111000
49	1	00110001	73	I	01001001	97	a	01100001	121	y	01111001
50	2	00110010	74	J	01001010	98	b	01100010	122	z	01111010
51	3	00110011	75	K	01001011	99	c	01100011	123	{	01111011
52	4	00110100	76	L	01001100	100	d	01100100	124	|	01111100
53	5	00110101	77	M	01001101	101	e	01100101	125	}	01111101
54	6	00110110	78	N	01001110	102	f	01100110	126	~	01111110
55	7	00110111	79	O	01001111	103	g	01100111	127		01111111

Кодировка символов

Кодировка ASCII (национальный алфавит)

128	Ç	144	É	160	á	176	▒	192	└	208	╨	224	α	240	≡
129	ü	145	æ	161	í	177	▒	193	┴	209	╤	225	ß	241	±
130	é	146	Æ	162	ó	178	▓	194	┬	210	╥	226	Γ	242	≥
131	â	147	ô	163	ú	179	│	195	├	211	╙	227	π	243	≤
132	ä	148	ö	164	ñ	180	┤	196	─	212	╘	228	Σ	244	⌠
133	à	149	ò	165	Ñ	181	╡	197	┼	213	╒	229	σ	245	⌡
134	å	150	û	166	ª	182	╢	198	╞	214	╓	230	µ	246	÷
135	ç	151	ù	167	º	183	╖	199	╟	215	╫	231	τ	247	≈
136	ê	152	ÿ	168	¿	184	╕	200	╚	216	╪	232	Φ	248	°
137	ë	153	Ö	169	⌐	185	╣	201	╔	217	┘	233	Θ	249	∙
138	è	154	Ü	170	¬	186	║	202	╩	218	┌	234	Ω	250	·
139	ï	155	¢	171	½	187	╗	203	╦	219	█	235	δ	251	√
140	î	156	£	172	¼	188	╝	204	╠	220	▄	236	∞	252	n
141	ì	157	¥	173	¡	189	╜	205	═	221	▌	237	φ	253	2
142	Ä	158	₧	174	«	190	╛	206	╬	222	▐	238	ε	254	■
143	Å	159	ƒ	175	»	191	┐	207	╧	223	▀	239	∩	255	

www.LookTables.com

Кодировка символов

Unicode

Unicode Transform Protocol (**UTF**) – алгоритм кодирования символов:

- **UTF-8 (код переменной длины; от 1 до 4 байт на символ)**
- **UTF-16 (код переменной длины; 2 или 4 байта на символ)**
- **UTF-32 (код постоянной длины; 4 байта на символ)**

Byte Order Mark (**BOM**) определяет, какой из протоколов используется.

Старшие байты UTF-8	Кодирование символа UTF-8
Старший бит 0	Для кодирования используется один байт
110	Для кодирования используются 2 байта ***110***10000 ***10***111100
1110	Для кодирования используются 3 байта ***1110***1000 ***10***001111 ***10***010101
11110	Для кодирования используются 4 байта ***11110***100 ***10***001111 ***10***110101 ***10***110101

Файловый ввод/вывод

Файловая переменная (дескриптор файла) – символьное обозначение места расположения файла.

Файловая переменная содержит:

1) Имя внешнего носителя, на котором расположен файл и путь к файлу на этом носителе.
2) Положение указателя позиции внутри файла
3) Указатель на буфер обмена
4) Режим доступа к данным.
5) Тип файла: текстовой или бинарный.

Файловая переменная создаётся при открытии файла.

После окончания работы с файлом его необходимо закрыть!

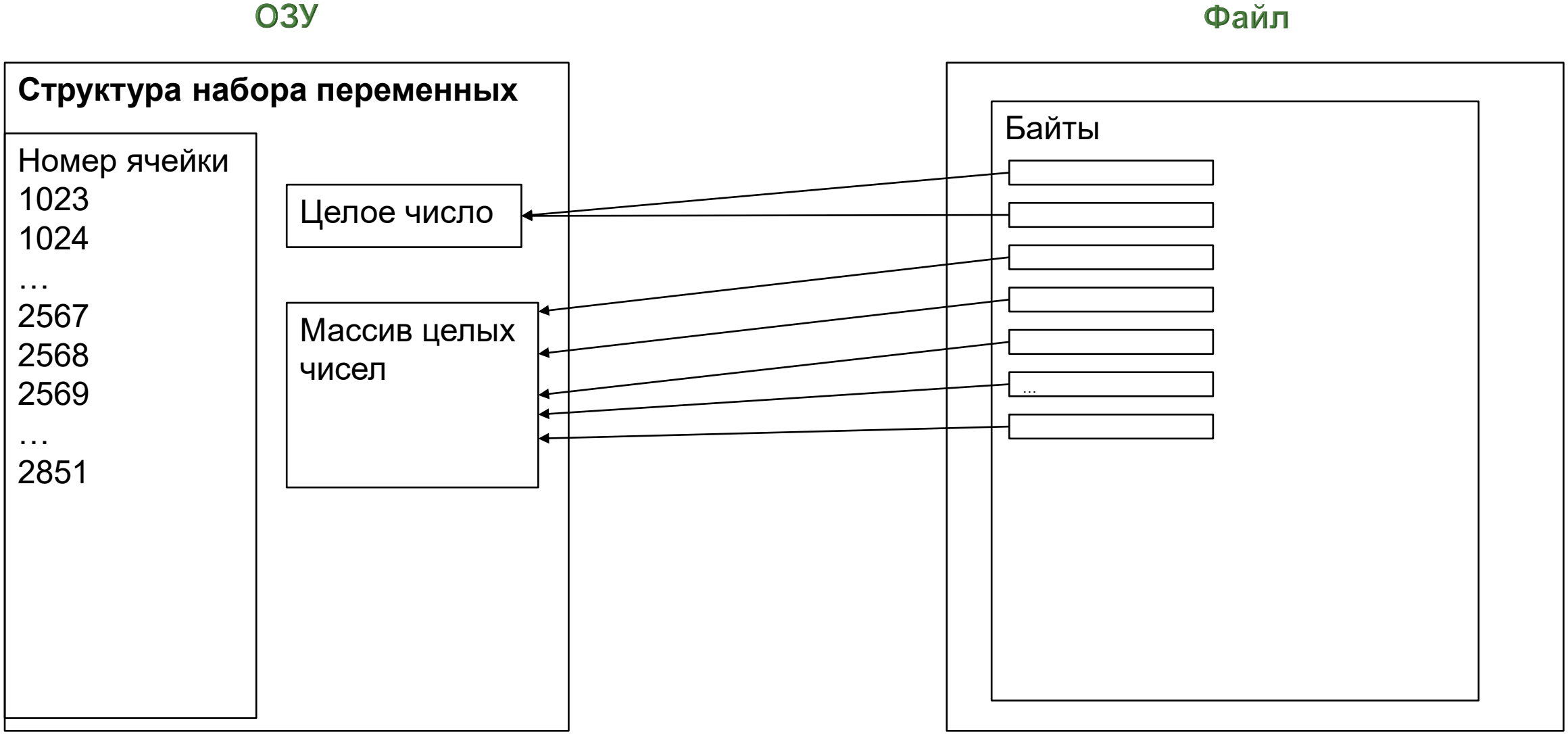
Файловый ввод/вывод (текстовой файл)

Единицы чтения данных из текстового файла:

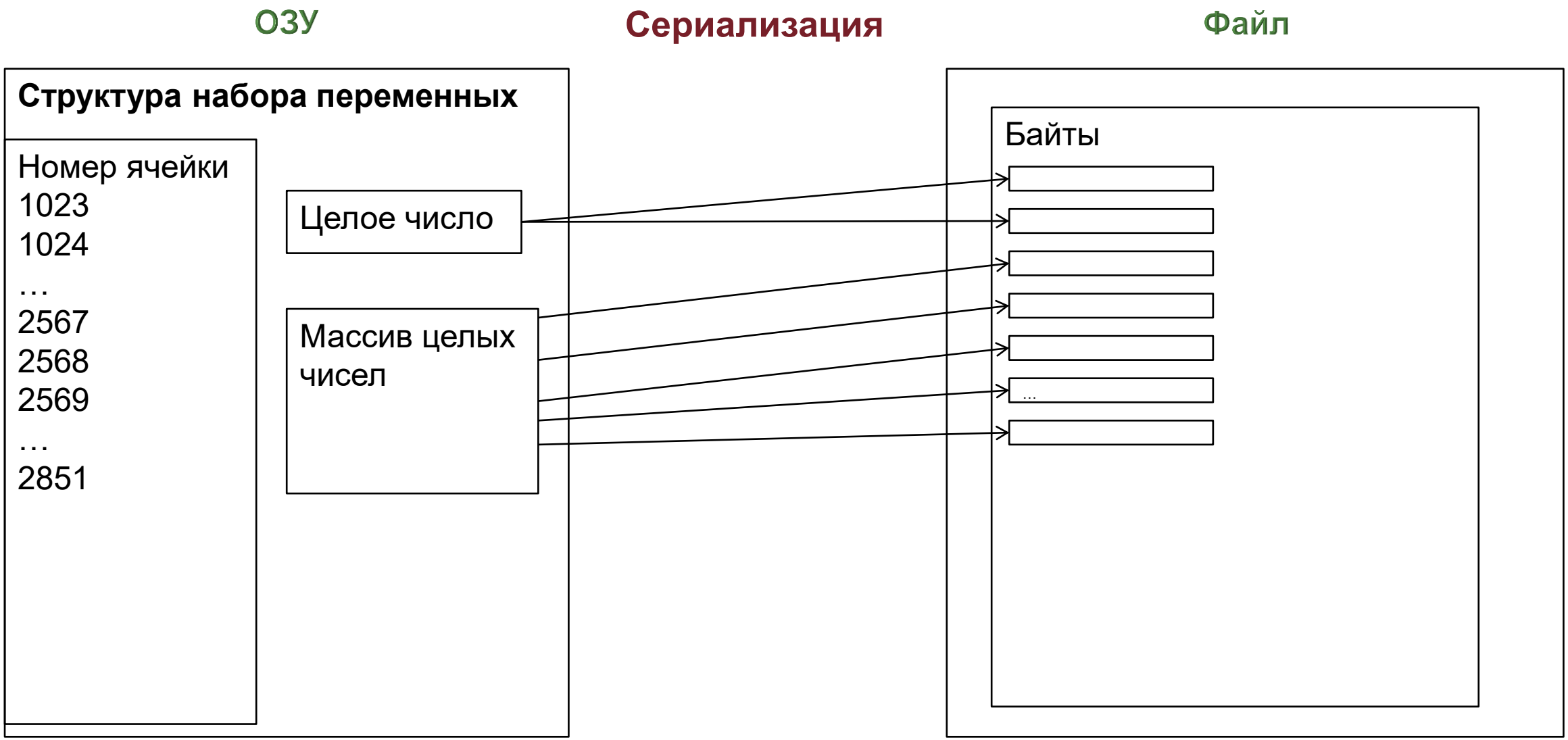
- символ;
- строка;
- файл целиком.

Единица записи данных в текстовой файл – строка.

Файловый ввод (бинарный файл)



Файловый вывод (бинарный файл)



Массив

Массив – это составная переменная, являющаяся совокупностью простых переменных **одинакового** типа.

Свойства массива:

1) все простые переменные, составляющие массив имеют одинаковый тип данных;
2) все простые переменные занимают в оперативной памяти один, непрерывный блок;
3) к каждой простой переменной имеется прямой доступ;
4) число простых переменных, составляющих массив, называется размерностью массива. Размерность массива определяется в момент его создания и в дальнейшем не изменяется.

Имя массива - переменная, обозначающая массив.
Имя массива ссылается на ячейку памяти, в которой расположен нулевой элемент массива.

Имя массива [номер элемента массива] – переменная, обозначающая элемент массива

Массив

ПРИМЕР

Имя массива AR

AR[0] – простая переменная, обозначающая нулевой элемент массива.

AR[50] – простая переменная, обозначающая пятидесятый элемент массива.

Y – простая переменная целого типа

При Y = 1

AR[Y] – простая переменная, обозначающая первый элемент массива.

При Y = 55

AR[Y] – простая переменная, обозначающая пятьдесят пятый элемент массива.

Массив, пример обработки массива

Задан массив AR целых чисел размерности 5. Необходимо присвоить вспомогательной переменной Z значение 1, если в массиве есть элемент со значением 77, и значение 2, если в массиве нет элемента со значением 77. Начальное значение переменной Z равно 0.

Номер ячейки оперативной памяти	Номер элемента массива AR	Имя переменной, обозначающей элемент массива AR
101	1	AR[1]
102		
103	2	AR[2]
104		
105	3	AR[3]
106		
107	4	AR[4]
108		
109	5	AR[5]
110		

```
1: Z := 0
2: FOR
3:     Заголовок
4:                 idx := 0
5:                 idx < 5
6:                 idx := idx + 1
7:     Конец заголовка
8:     Тело цикла
9:     IF AR[idx] = 77
10:                Z := 1
11:    Конец тела цикла
12: Конец FOR
13: IF Z = 0
14:    Z := 2
```

Строки символов

Строка символов – **массив** простых переменных типа CHAR.

Для строки вместо термина **«размер массива»** употребляется термин **«длина строки».**

Для выполнения операций со строками используются подпрограммы.

Типы операций со строками:

- *определение длины строки;*
- *сравнение строк;*
- *объединение строк;*
- *поиск подстроки в строке;*
- *преобразование строки в число.*

Строки символов

ПРИМЕР посимвольной обработки строки

Преобразовать в число строку, состоящей из последовательности цифр без знака и точки. Исходную строку обозначим S, результатом будет переменная X.

```
1: X := 0
2: L = len(S)
3: FOR
4:     Заголовок
5:                 idx := 0
6:                 idx < L
7:                 idx := idx + 1
8:     Конец заголовка
9:     Тело цикла
10:                X := X * 10 + ord(S[idx]) – ord('0')
11:    Конец тела цикла
12: Конец FOR
```

Список

Свойства списка:

1) все простые переменные, составляющие список, имеют одинаковый тип данных;
2) все простые переменные занимают в оперативной памяти один, непрерывный блок;
3) к каждой простой переменной имеется прямой доступ;
4) Начальная размерность списка определяется в момент его создания и может изменяться в случае необходимости.

Состояние списка определяется двумя переменными:
Cap – максимально возможное количество элементов в массиве;
Len – текущее количество элементов в массиве.

Стек, Дек, Очередь

Свойства стека, дека, очереди:

1) все простые переменные, составляющие составную переменную, имеют одинаковый тип данных;
2) все простые переменные занимают в оперативной памяти один, непрерывный блок;
3) Максимальная размерность составной переменной определяется в момент её создания и не может изменяться в дальнейшем.

Стек. Доступ разрешён только к первому элементу. Удалить можно только первый элемент. Новый элемент добавляется на первое место, для чего все остальные элементы сдвигаются на одну позицию.

Дек. Доступ разрешён к первому или последнему элементам. Удалить можно или первый или последний элемент. Новый элемент добавляется или на первое, или на последнее место. При добавлении на первое место, все существующие элементы сдвигаются на одну позицию.

Очередь. Доступ разрешён только к первому элементу. Удалить можно только первый элемент. Новый элемент добавляется на последнее место.

Структурированный тип данных

В структуру объединяются переменный различных типов.

Каждая из простых переменных в структуре называется полем.

Доступ к простой переменной в структуре осуществляется через имя структуры:
<имя структуры>.<имя поля>

ПРИМЕР

СТРУКТУРА Студент{
Фамилия: строковый тип;
Имя: строковый тип;
Отчество: строковый тип;
Дата рождения: тип «Дата»;
Номер группы: целочисленный тип;
Сумма стипендии: вещественный тип;
}

Группа - массив переменных типа Студент, длиной 35.

Имя студента с номером 3:
Группа[3].Имя.